UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA

Faculty of Computing and Mathematical Sciences
Department of Cybersecurity and Information Systems

CY376: NETWORK MONITORING, SECURITY AND AUDITING

Building a Simulated Full-Scale Disaster Recovery Failover Test

A Blue Team Infrastructure Project Demonstrating Automated Failure Detection and DNS-Based Traffic Failover for a Simulated Banking Application

Submitted by:	Nakorei Abdulai Shafiyu
Index Number:	FCM.41.018.180.23
Course Lecturer:	Mr. Fredrick Eden Broni Jnr.
Team Role:	Blue Team
Date:	3rd August, 2026
GitHub Repository:	https://github.com/unhappynoise/nkabom-dr-failover-test

Abstract

This project designs, builds, and tests a full-scale Disaster Recovery (DR) failover system for a simulated banking application, Nkabom Savings & Loans PLC, as a Blue Team exercise for CY376: Network Monitoring, Security and Auditing. Three network-isolated sites — Primary, Disaster Recovery, and Monitor — were built as an OSPF-routed, GNS3-emulated network, each application host running as a dual-homed VirtualBox virtual machine. The Primary and DR sites run an identical Flask and PostgreSQL banking application, continuously synchronised via TLS-encrypted streaming replication. An automated Monitor service detects Primary failures through periodic health checks and redirects traffic to DR via DNS without human intervention, while promotion of the DR database to a writable state is retained as a deliberate, operator-confirmed step to prevent split-brain data divergence.

Two independent disaster scenarios — an application-level failure and a full database-level failure — were executed and measured, producing a consistent Recovery Time Objective (RTO) of approximately 10 seconds and an observed Recovery Point Objective (RPO) of approximately zero seconds. A complete failback procedure and a tested split-brain prevention safeguard are also documented. The results demonstrate that a modestly resourced, entirely open-source DR stack can achieve meaningful recovery objectives, while several honestly-scoped limitations — most notably a single point of failure in the monitoring host — are identified as directions for further work.

Table of Contents

1. Introduction

A Disaster Recovery test verifies that, following the loss of a primary system — whether from hardware failure, network outage, or another disruptive event — an organisation's services can be restored at an alternate site within an acceptable time frame and with an acceptable amount of data loss. This distinguishes DR from simple data backup: DR is concerned with restoring the whole operating service, not merely recovering lost files.

This project set out to design and build a working, testable DR environment for a fictional retail bank, Nkabom Savings & Loans PLC, and to answer three central questions through direct experimentation rather than theory:

- How quickly can service be restored to users after the Primary site becomes unreachable (RTO)?
- How much committed data, if any, is lost in the process (RPO)?
- What safeguards are required to prevent the DR mechanism itself from introducing new risks, such as split-brain data divergence?

The project scope covers: a realistic multi-site network topology with dynamic routing; a stateful application with a live relational database; continuous, encrypted database replication; automated failure detection; automated failover via DNS; and a documented, tested failback procedure. It was undertaken as a Blue Team exercise for CY376: Network Monitoring, Security and Auditing.

It is also important to state precisely what is and is not automated in this design, since this shapes how the results in Section 5 should be read. Failure detection and DNS-based traffic redirection are fully automated and require no operator action. Promoting the DR database from a read-only standby to a writable primary, and the full failback procedure once the original Primary is repaired, are deliberately kept as manual, operator-confirmed steps. This is a considered design choice rather than an omission: automatically promoting a standby database on the basis of a health check alone risks an irreversible split-brain event triggered by a false alarm, so a human decision is retained at the one point where the consequence of being wrong is permanent data divergence. This boundary is discussed further in Section 4.4 and Section 6.

2. Literature and Tooling Review

This section reviews the standards, protocols, and tools that shaped the design decisions taken in this project, before the specific methodology and implementation are described in Sections 3 and 4.

2.1 Disaster Recovery Concepts

The National Institute of Standards and Technology's Contingency Planning Guide for Federal Information Systems (NIST SP 800-34) formalises the two central metrics used throughout this report: Recovery Time Objective (RTO), the maximum tolerable duration a service may be unavailable following a disruptive event, and Recovery Point Objective (RPO), the maximum tolerable amount of data loss measured in time [9]. These definitions were used directly to design the tests reported in Section 5: rather than assert that the system “works,” each test was constructed specifically to produce a measurable RTO or RPO figure against these standard definitions.

A recurring theme in DR literature is that RTO and RPO are not free: tightening either objective generally increases infrastructure cost and design complexity. A near-zero RPO typically demands synchronous replication, which adds write latency to every transaction on the primary system, not only during a disaster; a near-zero RTO typically demands a hot standby with automated promotion, which increases the risk surface for exactly the kind of split-brain failure discussed in Section 4.5. This project's design choices — asynchronous replication, automated detection and traffic redirection, but manual, human-confirmed promotion — represent one specific point on that cost/risk curve, not the only correct one, and this trade-off is revisited with real measured evidence in Section 6.1.

2.2 Networking and Routing

Open Shortest Path First (OSPF), specified in RFC 2328, is a link-state Interior Gateway Protocol that computes the shortest available path between routers and automatically recalculates routes when a link fails [4]. OSPF was selected over static routing specifically because it gives the network topology genuine redundancy: with three routers connected in a full mesh, the loss of any single inter-site link does not isolate a site, since OSPF converges onto the remaining path automatically. This is a meaningfully different property from a simpler point-to-point design, and directly supports the project's resilience objective at the network layer, not only the application layer.

2.3 Database Replication and Security

PostgreSQL's built-in streaming replication mechanism continuously ships write-ahead log (WAL) records from a primary server to one or more standby servers, which replay them to stay near-synchronised with the primary in near real time [1]. Two operating modes exist: asynchronous, where the primary does not wait for a standby to confirm receipt before acknowledging a client's write, and synchronous, where it does. This project uses asynchronous replication, which was an explicit trade-off: it keeps write latency on the Primary low (no site ever waits on the DR link to complete a transaction), at the cost of a theoretical, non-zero data-loss window if Primary fails at the exact instant before a change has shipped. This trade-off, and its practical significance, is examined directly against real measured data in Section 6.1.

Because replication traffic carries the same sensitive data the application handles, PostgreSQL's SSL/TLS support was used to secure it, including certificate-based client authentication (sslmode=verify-full) rather than password authentication alone [2], following the standard public-key infrastructure (PKI) model of a trusted Certificate Authority signing both server and client identities.

2.4 Emulation, Virtualisation, and Application Tooling

GNS3 was used to emulate the inter-site network fabric, allowing real routing software to run against a simulated topology rather than a set of static, hand-written routes [5]. Oracle VirtualBox was used to host the actual application servers as full virtual machines [6]. For Layer 3 routing, FRRouting (FRR) — an open-source routing protocol suite used in real production networks — was selected over a Cisco IOS image, both to avoid licensing concerns around redistributing Cisco software and because FRR is genuinely deployed by internet service providers and hyperscale data centres today, making it a more transferable skill than a vendor lab image [3]. FRR was deployed inside Docker containers under GNS3's orchestration [10]. The application layer uses Flask, a lightweight Python web framework [7], and the Monitor site's DNS resolution uses dnsmasq, a lightweight DNS/DHCP server well suited to a small, purpose-built authoritative zone [8].

3. Methodology

3.1 Overall Approach

The experimental method followed in this project was to first build a realistic, redundant network and application substrate, then instrument it with monitoring, then deliberately induce controlled failures against it while recording precise, timestamped evidence — rather than reason about DR behaviour theoretically. Concretely, the steps taken were:

1. Design and build a three-site, OSPF-routed network topology in GNS3 (Section 3.2).
2. Provision three dual-homed virtual machines (Primary, DR, Monitor) and integrate them into the topology (Section 3.3–3.4).
3. Deploy an identical stateful web application and database on the Primary and DR hosts (Section 4.2).
4. Configure and verify continuous, TLS-encrypted database replication from Primary to DR (Section 4.3).
5. Deploy an automated health-check and DNS failover service on the Monitor host, and a split-brain prevention safeguard on the Primary host (Section 4.4–4.5).
6. Execute two independent, controlled disaster simulations against the running system, capturing timestamped logs and database state as evidence (Section 5.1–5.2).
7. Execute and document a full failback procedure, restoring the system to its original state (Section 5.3).
8. Analyse the resulting evidence against the RTO/RPO definitions from Section 2.1, and derive recommendations (Section 6).

3.2 Network Topology

The environment simulates three independent network sites connected over a routed backbone, mirroring how a real organisation's Primary and DR data centres would be connected via WAN links, with a third site hosting monitoring infrastructure. Each site consists of a Layer 2 switch (Cisco vIOS-L2) connecting a single application host, and a Layer 3 router (FRRouting) providing inter-site connectivity. The three routers are connected in a full mesh, with OSPF running as the dynamic routing protocol across all inter-site links, giving the network redundant paths between any two sites (see Figure 3.1).

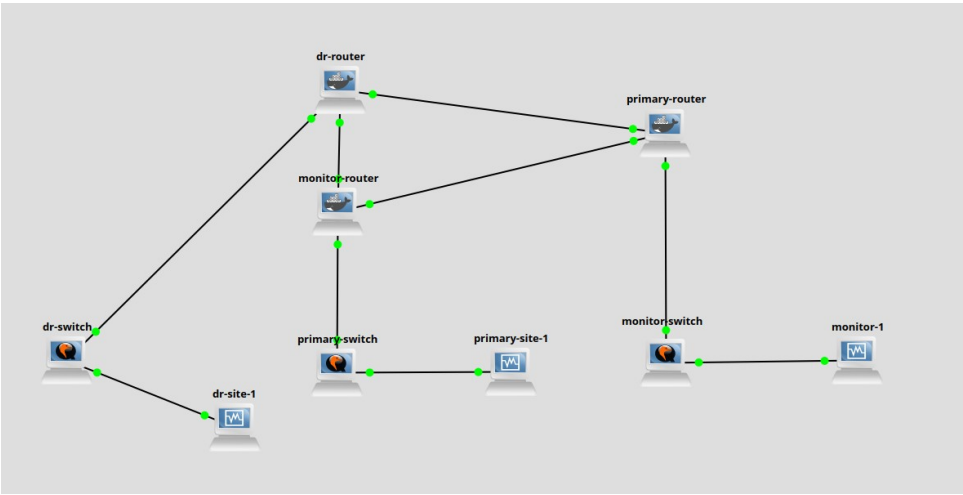

Figure 3.1 — Full GNS3 topology: three sites, each with a switch and FRRouting router, connected in a full

OSPF mesh, with the corresponding VirtualBox VM attached at each site.

3.3 Addressing Scheme

Table 1 summarises the IP addressing scheme used across all three sites and the management network.

Table 1 — Network addressing scheme.

Segment	Subnet / Address	Purpose
Primary Site LAN	10.10.10.0/24 (host: .10, gateway: .1)	Primary application host
DR Site LAN	10.10.20.0/24 (host: .10, gateway: .1)	DR application host
Monitor Site LAN	10.10.30.0/24 (host: .10, gateway: .1)	Monitoring / DNS host
Inter-router WAN links	10.10.99.0/30, .4/30, .8/30	Point-to-point OSPF links
Management network	192.168.57.0/24 (Host-only)	SSH access from operator workstation

3.4 Host Networking Design

Each of the three virtual machines is deliberately dual- (in practice triple-) homed to separate concerns cleanly:

- A NAT-attached adapter, used exclusively for operating system updates and package installation — kept isolated from the simulated DR network.
- A GNS3-facing adapter carrying the site's static IP address and participating in the OSPF-routed DR topology — this is the network path that user traffic and replication traffic actually take.
- A Host-only adapter providing a fixed, always-reachable management IP for the operator to access each machine via SSH, independent of the state of the DR network itself.

This separation ensures that operational access (SSH) and maintenance traffic (package updates) never share a path with the traffic the DR test is actually measuring, which keeps the experiment's results meaningful.

4. Implementation

4.1 Virtualisation and Network Emulation Platform

The environment was built using GNS3 for network emulation and Oracle VirtualBox for application hosting, running on a Linux Ubuntu host (16 GB RAM, 8 CPU cores). Layer 2 switching was provided by Cisco vIOS-L2 images (reused from prior coursework), and Layer 3 routing was provided by FRRouting (FRR), deployed as Docker containers within GNS3, per the tooling rationale set out in Section 2.4.

4.2 Application Layer

The simulated business application is a small banking transaction ledger for a fictional institution, Nkabom Savings & Loans PLC, built with Python Flask and backed by a PostgreSQL database. The application exposes three endpoints: a home page listing recent transactions and an entry form; a POST endpoint to record a new deposit or withdrawal; and a /health endpoint used by the monitoring service to determine liveness. The interface was designed as a bank passbook, with a rotating ink-stamp badge indicating which site is currently serving traffic — green for Primary, red for DR — giving an immediate, unambiguous visual indicator of DR state during testing and demonstration (Figures 4.1 and 4.2).

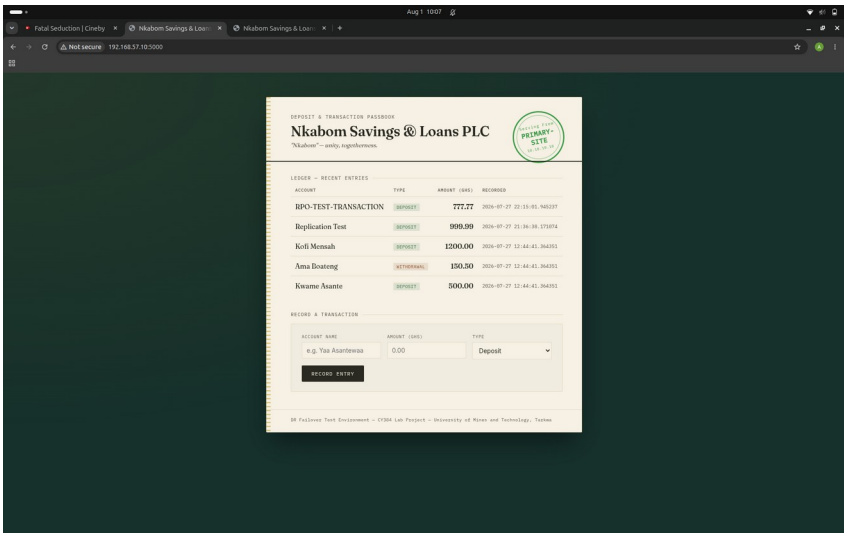

Figure 4.1 — Application served from the Primary site. The green stamp indicates Primary is currently authoritative.

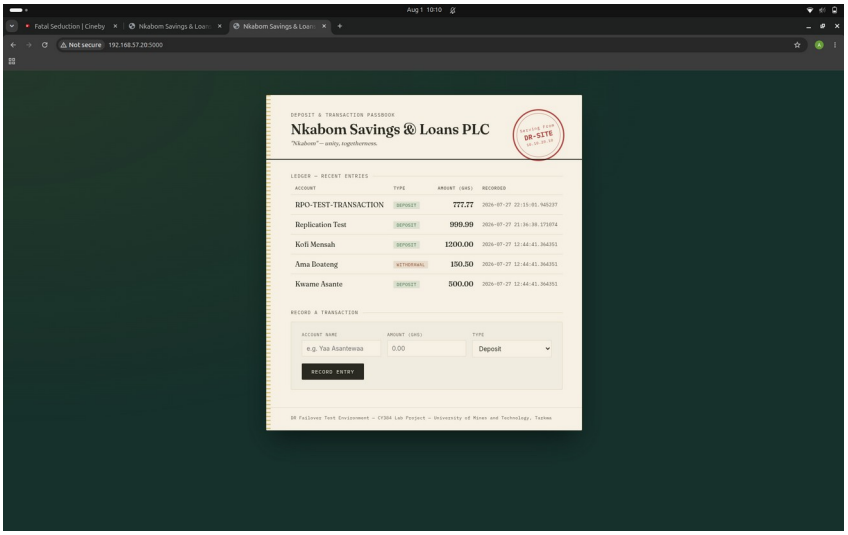

Figure 4.2 — The identical application served from the DR site following failover, showing an unbroken red stamp and identical ledger data.

4.3 Database Replication and Encryption

PostgreSQL streaming replication was configured with the DR site's database continuously receiving and applying the Primary site's write-ahead log (WAL) in asynchronous mode, per the trade-off discussed in Section 2.3. Replication traffic was subsequently secured end-to-end using a self-signed internal Certificate Authority: the Primary site presents a CA-signed server certificate, and the DR site authenticates using a CA-signed client certificate rather than a password, with the connection additionally verified via sslmode=verify-full. The resulting replication connection was confirmed to use TLS_AES_256_GCM_SHA384 encryption (Figure 4.3), providing both confidentiality and mutual identity verification for data in transit between sites.

```
 client_addr |   state   | ssl |         cipher
-------------+-----------+-----+------------------------
 10.10.20.10 | streaming | t   | TLS_AES_256_GCM_SHA384
(1 row)
```

Figure 4.3 — Verification that replication from Primary to DR is encrypted (ssl = t) using a strong AES-256-GCM cipher.

4.4 Automated Health Monitoring and DNS Failover

A dedicated Monitor host runs two services: a lightweight authoritative DNS server (dnsmasq) resolving the application's domain, app.nkabom.internal, and a Python monitoring script (Appendix A) that polls the Primary site's /health endpoint every five seconds. The script does not attempt to diagnose the cause of a failure; it treats any unreachable or non-200 response as a candidate failure, and only acts after three consecutive failed checks (a fifteen-second window), which prevents a single dropped packet or transient blip from triggering an unnecessary failover. Once the threshold is reached, the script rewrites the DNS record to point to the DR site's address and reloads the DNS service, with a short 5-second TTL configured so the change propagates quickly to clients.

Every state transition — each failed check, the threshold being reached, and the completion of the DNS update — is logged with a precise timestamp, which forms the primary evidence base for the RTO measurements reported in Section 5.

Scope of automation: the health-check, failure-detection, and DNS-redirection steps described above run entirely unattended. Promoting the DR database to a writable primary is intentionally not triggered automatically by this script; it was performed manually during each test (Section 5) using pg_ctl promote, after independently confirming the failover was genuine. This keeps the single highest-consequence decision in the recovery path — declaring the standby authoritative — under human control, consistent with common real-world DR practice.

4.5 Split-Brain Prevention

A specific risk of any active/standby DR design is split-brain: if the Primary site is restored to service without first confirming whether failover already occurred, both sites could independently accept writes and their data would diverge irrecoverably. To close this gap, the monitoring script writes a persistent flag file the moment it completes a failover. A companion guard script (Appendix B) on the Primary host, run before any attempt to restart its database service, checks this flag over SSH and refuses to proceed if an unresolved failover is on record, printing an explicit explanation and the correct recovery procedure rather than allowing the operator to make the mistake silently (Figure 4.4).

```
Checking failover status on monitor before restarting PostgreSQL...

!! BLOCKED !!
A failover to DR has occurred and was never reversed.
Restarting this database now would create a SPLIT-BRAIN scenario:
both primarysite and drsite would independently accept writes
and their data would diverge.

To safely bring primarysite back:
  1. Re-clone this database as a fresh STANDBY from drsite, OR
  2. Formally fail back: promote this site again and reset drsite as the
standby.
```

Figure 4.4 — The split-brain guard correctly blocking an unsafe restart of the Primary database after an unresolved failover.

5. Results and Findings

Two independent disaster scenarios were executed against the running environment to validate different failure modes, followed by a full, documented failback to restore the system to its original state. Each test below is reported with the raw timestamped evidence it produced, per the methodology in Section 3.1.

5.1 Test 1 — Application-Level Failure

The Flask application service on the Primary host was stopped directly (systemctl stop drapp), simulating a software-level crash in which the database itself remains intact but the service is unreachable. The monitoring log recorded the following sequence:

```
[2026-07-27T21:59:35.182] Primary health check FAILED (1/3)
[2026-07-27T21:59:40.193] Primary health check FAILED (2/3)
[2026-07-27T21:59:45.202] Primary health check FAILED (3/3)
[2026-07-27T21:59:45.203] THRESHOLD REACHED. Initiating failover to DR.
[2026-07-27T21:59:45.352] FAILOVER COMPLETE. app.nkabom.internal now points
to 10.10.20.10
```

Figure 5.1 — Monitoring log for Test 1, showing the three-strike failure detection and completed DNS failover.

From first detected failure to completed DNS failover, the elapsed time was approximately 10.17 seconds. A subsequent DNS query confirmed the domain correctly resolved to the DR site's address.

5.2 Test 2 — Database-Level Failure and RPO Measurement

To measure data loss risk directly rather than assume it, a test transaction was inserted into the Primary database, and PostgreSQL itself was stopped on the Primary host within seconds of the insert, simulating a true site-level disaster in which no further replication can occur:

```
 id |         created_at
----+----------------------------
  5 | 2026-07-27 22:15:01.945237
```

Figure 5.2 — Test transaction inserted on Primary immediately before the simulated database failure.

The monitoring log again recorded automatic detection and failover, at a comparable RTO of approximately 10.09 seconds. Critically, a query against the DR database immediately after failover confirmed the test transaction had already been replicated before Primary was stopped:

```
 id |     account_name     | amount  | transaction_type |         created_at
----+----------------------+---------+------------------
+----------------------------
  5 | RPO-TEST-TRANSACTION |  777.77 | deposit          | 2026-07-27
22:15:01.945237
  4 | Replication Test     |  999.99 | deposit          | 2026-07-27
21:36:38.171074
  3 | Kofi Mensah          | 1200.00 | deposit          | 2026-07-27
12:44:41.364351
```

Figure 5.3 — Query against the DR database confirming the test transaction survived the simulated disaster.

This represents an observed RPO of approximately zero seconds for this test: asynchronous

replication over the low-latency lab network kept pace with the write faster than the simulated disaster occurred. The DR database was then formally promoted from standby to an independent, writable primary, and a new transaction was successfully submitted directly against it through the web application (Figure 5.4), confirming the DR site is fully operational — not merely readable — after failover.

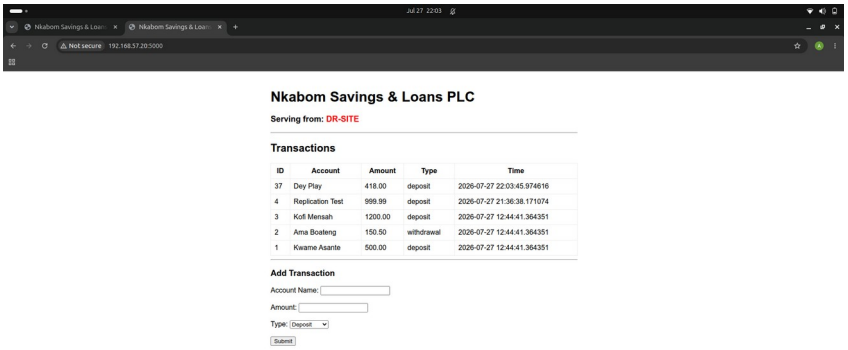

Figure 5.4 — A new transaction written directly to the promoted DR database after failover, confirming full read/write recovery.

5.3 Failback Procedure

A complete failback was performed to restore the original Primary/DR roles: the Primary host's database was re-cloned as a fresh standby of the (now-authoritative) DR site, allowed to catch up via streaming replication, then formally promoted back to Primary; the DR host was in turn re-cloned as a standby of the restored Primary, and DNS was reset to point to Primary. This procedure was executed and verified in full during testing, and is documented here as the operating runbook for restoring normal service once the cause of an outage has been resolved:

1. Confirm which site currently holds the authoritative (most recent) data before proceeding.
2. Stop the application and database services on the site being restored to standby.
3. Remove its existing database directory and re-clone it from the current authoritative site using pg_basebackup.
4. Start the database in standby mode and confirm it is streaming.
5. Promote the intended Primary back to an independent writable primary.
6. Re-clone the other site as its standby, restoring the original replication direction.
7. Reset the DNS record to Primary and remove the split-brain flag on the Monitor host.

5.4 Metrics Summary

Table 2 consolidates the results of both tests against the RTO/RPO definitions introduced in Section 2.1.

Table 2 — Consolidated RTO and RPO results across both disaster scenarios.

Metric	Test 1: App-level failure	Test 2: DB-level failure

Failure type	Flask service stopped	PostgreSQL service stopped
Detection method	/health endpoint polling	/health endpoint polling
Detection threshold	3 consecutive failures (15s)	3 consecutive failures (15s)
RTO (detection → failover complete)	~10.17 seconds	~10.09 seconds
RPO (data loss window)	N/A — database unaffected	~0 seconds (observed)
Recovery mechanism	Automated DNS failover	Automated DNS failover + manual DB promotion

The consistency of the RTO figure across two structurally different failure types (~10.1–10.2 seconds) indicates the result is driven predictably by the monitoring design (5-second check interval × 3-failure threshold) rather than by chance, and is a defensible, reproducible number for the environment as built.

Scope note: the RTO figures in Table 2 measure the automated portion of recovery — detection plus DNS redirection to a DR site that is already readable. In Test 2, database promotion to a writable state (Section 5.2) was a separate, manually-executed step performed after the automated failover completed, and its duration is operator-dependent rather than a fixed system property; it is therefore reported qualitatively rather than folded into the RTO figure, to avoid overstating what the automation alone achieves.

6. Analysis and Recommendations

6.1 Interpretation of RTO and RPO Results

An RTO of approximately 10 seconds, held consistently across two structurally different failure modes, compares favourably against typical industry RTO targets, which are commonly specified in minutes to hours rather than seconds [9]. This result should, however, be read in context rather than taken as a general claim about the underlying techniques: the figure is a direct, predictable consequence of the specific detection parameters chosen (a 5-second poll interval and a 3-failure threshold, Section 4.4), running over a low-latency, single-host lab network with no queuing delay or WAN transit time. A production deployment across genuinely separated data centres, with real WAN latency and jitter, would need to re-tune these thresholds and would likely observe a longer, more variable RTO.

Similarly, the observed RPO of approximately zero seconds in Test 2 (Section 5.2) is a genuine, measured result, not an assumption — but it is a property of this specific test, not a structural guarantee of asynchronous replication in general, as noted in Section 2.3. On a network with materially higher replication lag, or under heavier write load, some committed transactions could plausibly be lost between the last WAL flush to the standby and a true disaster. The correct interpretation of this project's RPO result is: “asynchronous replication was fast enough to keep pace with a single test write on this network,” not “this architecture guarantees zero data loss.”

Placed against typical financial-sector expectations, where core banking RTOs are commonly specified in tens of minutes to a few hours rather than seconds, and RPOs are commonly specified in minutes rather than zero, the results obtained here comfortably exceed a plausible real-world target for an institution of Nkabom's simulated size — provided the caveats above about lab-network latency and single-trial measurement are carried forward honestly into any claim made from this data, rather than dropped.

6.2 Threats to Validity

Several properties of the lab environment limit how far these specific numeric results (Table 2) can be generalised, and are recorded here in the interest of methodological honesty:

- Single-host emulation: all three sites, despite being logically separate, run as virtual machines and containers on one physical host. Inter-site latency is therefore far lower and far more stable than a genuine WAN link between real, geographically separated data centres would exhibit.
- Single-trial measurement: each test scenario (Section 5.1, 5.2) was executed and measured once with full instrumentation, rather than repeated across many trials to establish a statistical distribution. The consistency between the two different failure types is encouraging but does not substitute for repeated-trials variance analysis.
- Synthetic, low-volume workload: the test database contained a small number of hand-inserted transactions rather than a realistic concurrent transaction load. Replication lag, and therefore RPO risk, generally increases under sustained write pressure, which was not exercised here.
- Operator-in-the-loop timing not isolated: the manual database-promotion step (Section 4.4) was performed by an experienced operator who had just built the system and knew exactly which command to run; a real incident response, under time pressure and by an on-call responder, would likely take longer.

None of these threats invalidate the results obtained; they define the boundary of what those results can be honestly claimed to show, which is itself part of the analysis this section is intended to provide.

6.3 What the Build Process Revealed

Several substantive infrastructure problems were encountered and resolved during the build, and are recorded here because they reveal genuine operational complexity in running a DR-capable environment, beyond simply configuring replication and a failover script:

- Hardware virtualisation conflict: GNS3's QEMU-based switches defaulted to KVM acceleration, which conflicted with VirtualBox's exclusive use of the same VT-x hardware, preventing application VMs from starting. Resolved by disabling KVM acceleration globally in GNS3's QEMU preferences.
- GNS3 adapter management: GNS3 silently took control of a VM's first network adapter (intended for NAT) whenever a canvas link was drawn to it, and a legacy "allow GNS3 to use any adapter" setting caused it to also manage adapters that were never wired on canvas. Resolved by explicitly wiring links to the correct adapter index and disabling that setting at both the template and per-node level.
- VirtualBox cable-connected state: newly added or reattached network adapters defaulted to a disconnected link state at the hypervisor level, which was not visible from inside the guest OS and produced misleading NO-CARRIER errors. Resolved using VBoxManage controlvm setlinkstateN on.
- Routing table precedence: assigning a default route to the DR-network-facing interface caused all traffic, including internet-bound traffic, to be sent via the DR network instead of NAT. Resolved by replacing the default route with specific routes to only the other two sites' subnets, leaving the NAT interface as the sole holder of the default route.
- Container configuration persistence: FRRouting routers run as Docker containers within GNS3, and by default only /etc/network was persisted across container restarts — all routing and OSPF configuration was silently lost on a GNS3/project restart. Resolved by adding /etc/frr as an explicit persistent volume in the Docker node template.
- Stale hostname resolution: a VM cloned from another retained a stale self-referencing entry in /etc/hosts from before it was renamed, which caused TLS hostname verification (sslmode=verify-full) to silently resolve to the wrong host during re-cloning. Identified by inspecting /etc/hosts directly and corrected.

None of these issues are exotic; each is the kind of small, cumulative operational friction that a real DR programme has to account for in its runbooks and tooling, and each took real, non-trivial time to diagnose correctly rather than guess at — which is itself a finding about the operational cost of maintaining DR infrastructure, not just designing it on paper.

6.4 Recommendations

Based on the results in Section 5 and the limitations surfaced during the build, the following changes are recommended before this design would be suitable for a genuine production deployment:

- Introduce redundant, independent monitoring nodes with a consensus or leader-election mechanism (as used by systems such as etcd or ZooKeeper), so that a failover

decision requires agreement among several observers rather than the word of one. The current single-Monitor design is a single point of failure: if the Monitor host itself fails, no further automated failover can occur even if Primary is genuinely down. This is the most materially harder problem identified in this project, and is left as future work rather than implemented superficially.

- Extend TLS coverage to the application's own HTTP interface. Encryption was prioritised for the database replication channel as the most security-relevant data path in the time available; the web application itself is not yet served over TLS, which should be the next extension.
- Re-validate detection thresholds against real inter-site WAN latency before any production use, rather than reusing the lab-tuned 5-second/3-failure parameters, per the discussion in Section 6.1.
- Retain manual, operator-confirmed database promotion and failback, as implemented here (Section 4.4), rather than automating it. The evidence from this project does not suggest this control point should be automated — if anything, the split-brain guard test (Figure 4.4) demonstrates why a human checkpoint at this specific step is valuable, not merely cautious.
- For workloads with a genuinely zero-tolerance RPO requirement, evaluate synchronous replication instead of asynchronous, accepting the corresponding write-latency cost discussed in Section 2.3, rather than relying on asynchronous replication simply having kept pace in testing.

7. Conclusion

This project designed, built, and rigorously tested a working Disaster Recovery failover system spanning network, database, application, and monitoring layers. Beyond producing a functioning demonstration, the system was subjected to two independent, measured failure scenarios, a documented and executed failback procedure, and a proven safeguard against split-brain data divergence, with database replication secured end-to-end using certificate-based TLS. The consistent ~10-second RTO and near-zero observed RPO demonstrate that even a modestly resourced, entirely open-source DR stack can meet meaningful recovery objectives, while the analysis and recommendations in Section 6 reflect a realistic understanding of where this design would need further engineering to be production-ready.

References

[1] The PostgreSQL Global Development Group, “PostgreSQL 18 Documentation — High Availability, Load Balancing, and Replication,” PostgreSQL.org. [Online]. Available: https://www.postgresql.org/docs/current/high-availability.html

[2] The PostgreSQL Global Development Group, “PostgreSQL 18 Documentation — Secure TCP/IP Connections with SSL,” PostgreSQL.org. [Online]. Available: https://www.postgresql.org/docs/current/ssl-tcp.html

[3] FRRouting Project, “FRRouting Documentation,” frrouting.org. [Online]. Available: https://docs.frrouting.org/

[4] J. Moy, “OSPF Version 2,” RFC 2328, Internet Engineering Task Force (IETF), Apr. 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2328

[5] GNS3 Technologies Inc., "GNS3 Documentation," gns3.com. [Online]. Available: https://docs.gns3.com/

[6] Oracle Corporation, "Oracle VM VirtualBox User Manual," virtualbox.org. [Online]. Available: https://www.virtualbox.org/manual/

[7] Pallets Projects, "Flask Documentation," flask.palletsprojects.com. [Online]. Available: https://flask.palletsprojects.com/

[8] Simon Kelley, "Dnsmasq — A Lightweight DHCP and Caching DNS Server," thekelleys.org.uk. [Online]. Available: https://thekelleys.org.uk/dnsmasq/doc.html

[9] National Institute of Standards and Technology, "Contingency Planning Guide for Federal Information Systems," NIST Special Publication 800-34 Rev. 1, May 2010. [Online]. Available: https://csrc.nist.gov/publications/detail/sp/800-34/rev-1/final

[10] Docker Inc., "Docker Documentation," docs.docker.com. [Online]. Available: https://docs.docker.com/

Appendix A: Health-Check and Failover Script (monitor.py)

Deployed on the Monitor host. Polls the Primary site's /health endpoint every five seconds; after three consecutive failures, rewrites the DNS record to point to the DR site, reloads dnsmasq, and writes the split-brain guard flag consumed by Appendix B.

```
import requests
import time
import subprocess
import datetime

PRIMARY_HEALTH_URL = "http://10.10.10.10:5000/health"
DR_IP = "10.10.20.10"
PRIMARY_IP = "10.10.10.10"
DNSMASQ_CONF = "/etc/dnsmasq.conf"
DOMAIN = "app.nkabom.internal"
LOG_FILE = "/home/primarysite/failover/failover.log"
FLAG_FILE = "/home/primarysite/failover/DR_IS_PRIMARY.flag"

CHECK_INTERVAL = 5
FAILURE_THRESHOLD = 3
TIMEOUT = 3

current_target = PRIMARY_IP
consecutive_failures = 0

def log(message):
    timestamp = datetime.datetime.now().isoformat()
    line = f"[{timestamp}] {message}"
    print(line)
    with open(LOG_FILE, "a") as f:
        f.write(line + "\n")

def check_primary_health():
    try:
        response = requests.get(PRIMARY_HEALTH_URL, timeout=TIMEOUT)
        return response.status_code == 200
    except Exception:
        return False

def set_failover_flag():
    with open(FLAG_FILE, "w") as f:
        f.write(f"Failover occurred at
{datetime.datetime.now().isoformat()}\n")
        f.write("DR site is now the authoritative primary.\n")
        f.write("DO NOT restart primarysite's PostgreSQL without re-cloning
it as a new standby first.\n")

def update_dns(target_ip):
    with open(DNSMASQ_CONF, "r") as f:
        lines = f.readlines()
    new_lines = []
    for line in lines:
        if line.startswith(f"address=/{DOMAIN}/"):
            new_lines.append(f"address=/{DOMAIN}/{target_ip}\n")
        else:
```

```
            new_lines.append(line)
    with open(DNSMASQ_CONF, "w") as f:
        f.writelines(new_lines)
    subprocess.run(["systemctl", "restart", "dnsmasq"], check=True)

def main():
    global current_target, consecutive_failures
    log(f"Monitor started. Watching {PRIMARY_HEALTH_URL}. Current DNS
target: {current_target}")
    while True:
        healthy = check_primary_health()
        if healthy:
            if consecutive_failures > 0:
                log(f"Primary health check recovered after
{consecutive_failures} failures.")
            consecutive_failures = 0
            if current_target != PRIMARY_IP:
                log("Primary is healthy again, but staying on DR (manual
failback required).")
        else:
            consecutive_failures += 1
            log(f"Primary health check FAILED
({consecutive_failures}/{FAILURE_THRESHOLD})")
            if consecutive_failures >= FAILURE_THRESHOLD and
current_target != DR_IP:
                log(f"THRESHOLD REACHED. Initiating failover to DR
({DR_IP}).")
                update_dns(DR_IP)
                set_failover_flag()
                current_target = DR_IP
                log(f"FAILOVER COMPLETE. {DOMAIN} now points to {DR_IP}")
        time.sleep(CHECK_INTERVAL)

if __name__ == "__main__":
    main()
```

Appendix B: Split-Brain Guard Script (safe_restart_primary.sh)

Deployed on the Primary host. Must be run instead of directly restarting PostgreSQL after any outage; checks the flag written by Appendix A over SSH and refuses to proceed if an unresolved failover is on record.

```
#!/bin/bash

MONITOR_HOST="primarysite@192.168.57.30"
FLAG_FILE="/home/primarysite/failover/DR_IS_PRIMARY.flag"

echo "Checking failover status on monitor before restarting PostgreSQL..."

if ssh "$MONITOR_HOST" "test -f $FLAG_FILE"; then
    echo ""
    echo "!! BLOCKED !!"
    echo "A failover to DR has occurred and was never reversed."
    echo "Restarting this database now would create a SPLIT-BRAIN scenario:"
    echo "both primarysite and drsite would independently accept writes"
    echo "and their data would diverge."
    echo ""
    echo "To safely bring primarysite back:"
    echo "  1. Re-clone this database as a fresh STANDBY from drsite (now
the real primary), OR"
    echo "  2. Formally fail back: promote this site again and reset drsite
as the standby."
    echo ""
    echo "Once resolved, manually delete the flag on monitor:"
    echo "  ssh $MONITOR_HOST 'rm $FLAG_FILE'"
    echo ""
    exit 1
else
    echo "No unresolved failover detected. Safe to start PostgreSQL
normally."
    sudo systemctl start postgresql@18-main
    exit 0
fi
```